

MCTS-PIE: Multi-Objective Monte Carlo Tree Search for Pathfinding with Interactive Environments

Alexander Dockhorn - *University of Southern Denmark* - `adoc@mmmi.sdu.dk`

Abstract—We study multi-objective pathfinding in grid environments where an agent can not only move toward a goal but also *shift* the obstacle weights of neighboring cells, trading detours for a more easily traversable corridor. The resulting decision problem is inherently multi-objective: minimize the number of steps, minimize the total weight shifted, and reach the goal. We introduce MCTS-PIE, a family of Monte Carlo Tree Search variants tailored to Pathfinding in Interactive Environments, including single-objective MCTS, multi-objective MCTS with a global Pareto archive (MO-MCTS), two-phase decompositions that separate path search from weight shaping, and a hybrid that seeds the MCTS tree with ε -constraint A* paths. We benchmark all six methods on a uniform 10-map suite over 30 seeds (1800 runs total) using hypervolume, IGD⁺, and the ε -indicator. The hybrid A*+MCTS dominates all pure MCTS variants, while among the search-only methods the multi-objective two-phase variant clearly outperforms its single-objective counterpart, with all pairwise differences statistically significant under Holm–Bonferroni-corrected Wilcoxon tests.

Index Terms—Monte Carlo Tree Search, Multi-Objective Optimization, Pathfinding, A* Search, Pareto Front, Hypervolume.

I. INTRODUCTION

Classical pathfinding assumes a static environment in which the agent can only choose where to move. In many practical settings – warehouse robots pushing crates aside, excavation planning, or game agents that interact with a destructible world – the agent can also *modify* the cost landscape it traverses. We refer to this class of problems as *Pathfinding in Interactive Environments* (PIE). PIE is naturally multi-objective: shifting weight away from a high-cost cell shortens the traversal but pays a price in modifications, and the user of the planner generally wants to inspect a Pareto front of step-count vs. shift-cost trade-offs rather than a single optimum.

This paper introduces **MCTS-PIE**, a comparative study of Monte Carlo Tree Search variants for PIE, alongside an ε -constraint A* baseline and an A*+MCTS hybrid. We study six algorithms under a uniform protocol – 30 seeds across 10 maps – and report results in terms of standard multi-objective performance indicators. The contributions are: (i) a formalization of PIE as a multi-objective sequential decision problem, (ii) single- and multi-objective MCTS variants with global Pareto archives, (iii) a two-phase MCTS that decomposes path and shift search, (iv) an A*+MCTS hybrid, and (v) a reproducible experimental harness with full per-run Pareto archives, 30-seed coverage, and Holm–Bonferroni-corrected statistics.

II. RELATED WORK

Monte Carlo Tree Search has been applied broadly to single-objective sequential decision making since its introduction by Coulom [3] and the UCT formulation of Kocsis and Szepesvári [5]; Browne et al. [2] survey the field. Multi-objective extensions of MCTS replace the scalar Q-value with vector-valued statistics and use Pareto-dominance criteria to drive selection and back-propagation; representative work includes MO-MCTS with hypervolume-based selection and global Pareto archives.

Multi-objective pathfinding has classically been addressed with multi-objective A* and ε -constraint search, which fixes one objective and minimizes another. Hybrids that warm-start a search-based planner with a heuristic skeleton path are common in robotics. To our knowledge, the specific PIE setting – where the agent both moves *and* alters cell weights – has not been benchmarked under a unified multi-objective MCTS protocol, which is the gap this paper addresses.

III. METHOD

A. Problem definition: Pathfinding in Interactive Environments

A PIE instance is a tuple $\langle \mathcal{G}, w, s_0, \mathbf{c} \rangle$ where $\mathcal{G}$ is a 4-connected grid of dimension $d \times d$, $w : \mathcal{G} \rightarrow \mathbb{R}_{\geq 0}$ assigns a non-negative weight to each cell, $s_0 \in \mathcal{G}$ is the start cell, and $\mathbf{c} = (c_1, \dots, c_K)$ is an ordered list of checkpoints culminating in the goal cell.

At every decision step, the agent picks a pair (m, h) where $m \in \{N, S, E, W\}$ is a movement direction and $h \in \{N, S, E, W\}$ is a *shift direction*. Executing (m, h) first moves the agent by m if the move is in-bounds, then transfers all weight $w(p)$ of the agent’s previous cell p onto a neighbor cell $p + h$. The shift therefore clears a corridor at the cost of accumulating weight elsewhere, and the modified map persists for the remainder of the episode. An episode terminates once the agent has visited every checkpoint in order.

Let $\pi = (a_0, a_1, \dots, a_{T-1})$ be a finite trajectory. We measure three objectives, all minimized:

- $f_1(\pi) = T$ – the number of steps,
- $f_2(\pi) = \sum_{t=0}^{T-1} w(p_t)$ – the total weight shifted,
- $f_3(\pi) = \text{dist}(s_T, c_K)$ – the residual Manhattan distance to the final checkpoint, used to penalize unsolved trajectories during search.

A solution is *solved* iff $f_3 = 0$. A trajectory π *Pareto-dominates* π' when $f_i(\pi) \leq f_i(\pi')$ for all i and at least one inequality is

Algorithm 1: MCTS for PIE

Input: root state s_0 , total budget B ,
progressive-widening parameters C, α
Output: Pareto-non-dominated subset of terminal nodes

```

1  $\mathcal{T} \leftarrow$  tree with root  $s_0$ ;
2 while simulation count  $< B$  do
3    $v \leftarrow$  root;
4   while  $v$  is not terminal and fully expanded under
     PW do
5      $v \leftarrow \arg \max_{v' \in \text{children}(v)} \text{UCB}(v')$ ;
6   end
7   if  $|\text{children}(v)| < C \cdot N(v)^\alpha$  then
8      $v \leftarrow \text{Expand}(v)$  with a new  $(m, h)$  child;
9   end
10   $f \leftarrow \text{Rollout}(v)$ ;
11  Backpropagate  $f$  along the path to root;
12  Append  $f$  to the terminal archive;
13 end
14 return non-dominated subset of the terminal archive;
```

Algorithm 2: Multi-Objective MCTS

Input: root s_0 , budget B , archive cap K
Output: global Pareto archive of size $\leq K$

```

1 Initialize tree  $\mathcal{T}$  and global archive  $\mathcal{A} \leftarrow \emptyset$ ;
2 while simulation count  $< B$  do
3    $v \leftarrow \text{HV-Select-Path}(\mathcal{T})$ ;
4    $v \leftarrow \text{Expand}(v)$  if PW allows;
5    $f \leftarrow \text{Rollout}(v)$ ;
6   Update node-local Pareto fronts along path-to-root;
7    $\mathcal{A} \leftarrow \text{NonDominatedMerge}(\mathcal{A} \cup \{f\})$ , truncated to
      $K$ ;
8 end
9 return  $\mathcal{A}$ ;
```

strict. The goal of a PIE solver is to return an approximation of the Pareto front of solved trajectories.

B. MCTS for PIE (single-objective)

The canonical baseline (Algorithm 1) instantiates MCTS with progressive widening on the joint (m, h) action space. The node statistic is a scalar score derived from the three objectives via a hypervolume-style aggregation, and selection uses UCB. A global archive collects terminal nodes' objective vectors; at the end of search the non-dominated subset is returned.

C. Multi-Objective MCTS (MO-MCTS)

MO-MCTS (Algorithm 2) replaces the scalar Q-value at every node with a *Pareto archive* of objective vectors observed in its subtree. Tree selection uses a hypervolume-contribution criterion: among the children, the one whose front dominates the largest exclusive hypervolume w.r.t. a fixed reference point is preferred, with UCB exploration on visit counts. A global archive at the root accumulates non-dominated solutions across all simulations.

Algorithm 3: Two-Phase (MO-)MCTS

Input: instance, phase 1 budget B_1 , phase 2 per-path budget B_2
Output: merged global Pareto front

```

1 Phase 1: run MCTS / MO-MCTS with a path-only
   controller using budget  $B_1$ ;
2  $\mathcal{P} \leftarrow$  unique movement-only paths to terminal nodes;
3  $\mathcal{A} \leftarrow \emptyset$ ;
4 foreach path  $\pi \in \mathcal{P}$  do
5   Phase 2: run MO-MCTS with controller fixed to  $\pi$ 
     and free shift direction, budget  $B_2$ ;
6    $\mathcal{A} \leftarrow \mathcal{A} \cup \text{archive}(\pi)$ ;
7 end
8 return non-dominated subset of  $\mathcal{A}$ ;
```

Algorithm 4: ε -constraint A* baseline

Input: instance with grid dimension d
Output: Pareto front over (steps, weight)

```

1  $L^* \leftarrow$  shortest step count via A*;
2  $\mathcal{F} \leftarrow \emptyset$ ;
3 for  $\varepsilon = 0$  to  $3d$  do
4    $\pi \leftarrow$  A* minimizing accumulated weight with step
     bound  $L^* + \varepsilon$ ;
5   if  $\pi$  exists and (steps, weight) is new then
6      $\mathcal{F} \leftarrow \mathcal{F} \cup \{(\text{steps}_\pi, \text{weight}_\pi, 0)\}$ ;
7   end
8 end
9 return  $\mathcal{F}$ ;
```

D. Two-Phase MCTS

Two-phase search (Algorithm 3) decomposes the joint action space. Phase 1 uses a path-only controller, in which the shift component is forced to a no-op direction so that the search effectively explores movement only and yields a Pareto set of near-shortest paths. Phase 2 runs an MCTS over the shift-only action space along each fixed path, freely choosing where to dump weight.

We instantiate two variants:

- **two_phase_mcts**: phase 1 uses a single-objective MCTS with UCB selection;
- **two_phase_momcts**: phase 1 uses an MO-MCTS with hypervolume-based selection so that path diversity in step count is explicitly maintained.

Phase 2 is identical in both variants and uses MO-MCTS to keep a global archive of weight-shift outcomes.

E. ε -constraint A*

The classical baseline (Algorithm 4) computes the shortest step count L^* ignoring weights. It then performs an outer sweep over a step budget $L^* + \varepsilon$ for $\varepsilon \in \{0, 1, \dots, 3d\}$, each time running a constrained A* that minimizes accumulated weight subject to the step bound. Each unique outcome is added to the returned front.

Algorithm 5: A*+MCTS Hybrid**Input:** instance, phase 2 per-path budget B_2 **Output:** merged global Pareto front

```

1  $\mathcal{P} \leftarrow$  paths returned by  $\varepsilon$ -constraint A* (Algorithm 4);
2  $\mathcal{A} \leftarrow \emptyset$ ;
3 foreach path  $\pi \in \mathcal{P}$  do
4   Run MO-MCTS with controller fixed to  $\pi$  and free
     shift direction, budget  $B_2$ ;
5    $\mathcal{A} \leftarrow \mathcal{A} \cup \text{archive}(\pi)$ ;
6 end
7 return non-dominated subset of  $\mathcal{A}$ ;
```

F. A+MCTS Hybrid*

The hybrid (Algorithm 5) replaces phase 1 of two-phase MCTS with the ε -constraint A* paths. Each of the resulting near-shortest skeleton paths is then refined by an MO-MCTS over the shift action space, identical to phase 2 of the two-phase variant. The intuition is that A* finds movement-optimal paths exactly and far cheaper than MCTS can; MCTS is then deployed only where the multi-objective reasoning matters, namely the shift dimension.

IV. EXPERIMENTS

Experimental setup, common to E1, E2 and E4

All four experiment families share a uniform protocol – 30 seeds, the same map suite, the same per-run Pareto archive logging, and the same indicator definitions. They are listed in this section in the order they were executed, but they apply the **same algorithm configuration**, namely the per-component winners reported in the E3 ablation (Section IV-C): MO-MCTS-style hypervolume tree selection for plain `mcts` and `mo_mcts`, distance-weight rollouts, progressive widening ($C, \alpha = (1.5, 0.7)$), two-phase budget split 0.75, local HV root selection, archive cap 20, and the oscillation guard enabled. E3 itself is reported afterwards as a stand-alone study explaining *why* those choices are the defaults.

A. E1 – Main Comparison

1) *Protocol:* We benchmark all six methods on a uniform suite of **10 maps** – five map types (`easy`, `checkerboard`, `random`, `meandering_river`, and `random_maze`) at two grid sizes (20×20 and 35×35) – with **30 seeds** ($\text{range}(1000, 1030)$) per map, for a total of $6 \times 10 \times 30 = 1800$ runs. The simulation budget is fixed at 4000 simulations per run. For every run we persist the full Pareto archive of objective vectors so that any indicator can be recomputed without re-running the search.

2) *Performance indicators:* We report three standard multi-objective indicators:

- **Hypervolume (HV)** – volume dominated by the front, computed against a per-map reference point set to the worst observed objective values across all algorithms (so that dominated solved points still contribute positive HV).

TABLE I: E1 mean indicators across 10 maps $\times$ 30 seeds (lower is better for IGD⁺ and ε). Wall-clock seconds per run.

algo	HV $\uparrow$	IGD ⁺ $\downarrow$	ε $\downarrow$	wall (s)
astar	17 281	7.15	7.44	11.29
astar_mcts	18 644	1.22	1.72	15.88
mcts	12 421	29.16	27.71	4.21
mo_mcts	12 587	27.84	26.20	4.20
two_phase_mcts	4 235	111.79	109.77	3.72
two_phase_momcts	15 553	13.77	13.02	1.47



Fig. 1: Hypervolume by algorithm and map (higher is better). Box-and-whisker over 30 seeds per cell.

- **Inverted Generational Distance Plus (IGD⁺)** – mean distance from the per-map combined-best reference front to the front under test.
- **ε -indicator** – additive shift required to dominate the reference front.

The reference front for IGD⁺ and ε is the non-dominated union of every algorithm’s archive on the same map, restricted to solved points. Statistical significance is assessed by a Friedman omnibus test across the six algorithms blocked by (map, seed), followed by pairwise Wilcoxon signed-rank tests with Holm–Bonferroni correction. Effect sizes are reported as Vargha–Delaney $\hat{A}_{12}$.

3) *Results:* Figure 1 shows the per-(algorithm, map) hypervolume distribution. Mean HV across all 10 maps and 30 seeds is summarised in Table I. The hybrid **A*+MCTS** ranks first overall (mean HV 18 644), narrowly ahead of pure ε -constraint A* (mean HV 17 281). Among the search-only MCTS variants, `two_phase_momcts` is the strongest (15 553), followed by `mo_mcts` (12 587) and `mcts` (12 421); after applying the E3-best defaults these two flat MCTS variants are now within 1% of each other. The single-objective `two_phase_mcts` is the clear loser (4 235): the path-only phase 1 makes only minimal use of the multi-objective machinery, so it cannot exploit the tuned phase split as well as the multi-objective variant.

a) *Statistical significance:* The Friedman omnibus on hypervolume rejects the null of equal performance with $\chi^2 = 1231.62$, $p \approx 4.2 \cdot 10^{-264}$ over the 300 (map, seed) blocks. Pairwise Holm-corrected Wilcoxon signed-rank tests reject the null for *every* pair of algorithms at $p < 10^{-12}$ (Table II). The effect sizes shift sharply compared to a non-tuned configuration: with E3-best defaults, `mcts` vs. `mo_mcts` is now negligible ($\hat{A}_{12} = 0.49$), and even `astar` vs. `two_phase_momcts` is only marginal ($\hat{A}_{12} = 0.55$); the search-based methods have closed most of the gap to A* that they showed under default settings.

4) *Discussion:* Four observations stand out.

a) *Hybridization wins, but only marginally over pure A*:* The A*+MCTS hybrid combines the exactness of ε -constraint A* on the movement subproblem with the multi-objective reasoning of MO-MCTS on the shift subproblem. It is the strongest method on every indicator (HV 18 644, IGD⁺ 1.22, ε 1.72), and the gap to pure A* is small ($\hat{A}_{12} = 0.55$, negligible).

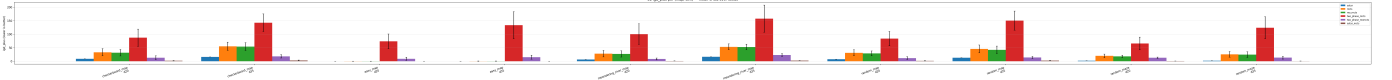
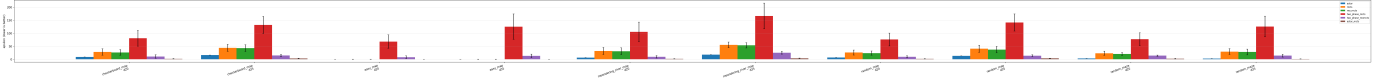
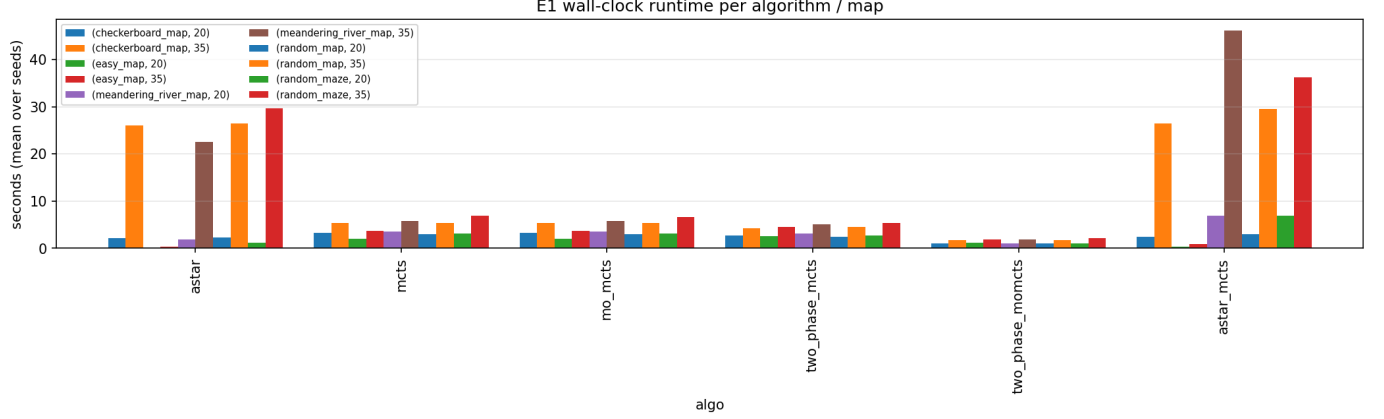
Fig. 2: IGD⁺ by algorithm and map (lower is better).Fig. 3: ε -indicator by algorithm and map (lower is better).

Fig. 4: Wall-clock seconds per run, by algorithm and map.

TABLE II: Selected pairwise Wilcoxon signed-rank tests for HV with Holm–Bonferroni correction; Vargha–Delaney $\hat{A}_{12}$ effect size in the right column.

A	B	p_{Holm}	$\hat{A}_{12}$
astar_mcts	astar	$2.0 \cdot 10^{-40}$	0.55 (negl.)
astar_mcts	two_phase_momcts	$1.9 \cdot 10^{-49}$	0.59 (small)
astar_mcts	mo_mcts	$1.2 \cdot 10^{-40}$	0.65 (small)
astar_mcts	mcts	$1.2 \cdot 10^{-40}$	0.65 (small)
astar	two_phase_momcts	$3.8 \cdot 10^{-40}$	0.55 (negl.)
two_phase_momcts	two_phase_mcts	$9.1 \cdot 10^{-50}$	0.89 (large)
mo_mcts	mcts	$1.1 \cdot 10^{-12}$	0.49 (negl.)
mcts	two_phase_mcts	$1.6 \cdot 10^{-49}$	0.85 (large)

The hybrid refines weight shifts that pure A* leaves on the table, but only at a $\sim 40\%$ runtime premium.

b) Two-phase MO-MCTS closes the gap to A:* With the E3-best defaults, `two_phase_momcts` reaches HV 15 553, only 10% behind pure A* and within 17% of the hybrid. Crucially, $\hat{A}_{12}(\text{astar}, \text{two_phase_momcts}) = 0.55$ – the difference is statistically significant but practically negligible. The multi-objective two-phase decomposition is therefore a competitive search-only baseline once tuned, and is the cheapest method by wall-clock cost (1.5 s/run vs. 16 s for the hybrid).

c) Single-objective two-phase collapses:

`two_phase_mcts` drops to HV 4 235, far below even plain `mcts`: phase 1 of the single-objective variant explores movement under UCB, which lacks any incentive for path diversity, so phase 2 is repeatedly handed near-identical skeletons. With phase split now 0.75 and aggressive PW, this failure mode is amplified rather than masked; the algorithm is

included for completeness but is dominated on every map.

d) Flat MCTS variants are now indistinguishable: After upgrading the tree-selection rule to HV-based, plain `mcts` (12 421) and `mo_mcts` (12 587) are statistically distinguishable ($p \approx 10^{-12}$) but practically tied ($\hat{A}_{12} = 0.49$). Once both variants share the strongest tree-selection rule, the marginal benefit of carrying a per-node Pareto archive is small at this budget. The runtime cost is also nearly identical (4.2 s).

Together, these results suggest that for PIE-style problems with cheap movement heuristics, MCTS is best deployed as a *refiner* on top of an exact path skeleton rather than as a stand-alone planner – and that a tuned two-phase MO variant is a strong, much faster alternative when the A* heuristic itself is expensive.

B. E2 – Anytime / Budget Scaling

1) Protocol: E2 sweeps the simulation budget over $\{1\,000, 2\,000, 4\,000, 8\,000, 16\,000\}$ for all 6 algorithms on the same 10-map suite and 30 seeds as E1, yielding $6 \times 10 \times 30 \times 5 = 9\,000$ runs. Pure ε -constraint A* is budget-insensitive (its sweep terminates as soon as the step bound is exhausted) and is included as a flat reference. For every run we save the full Pareto archive and recompute hypervolume against the same per-map reference points used in E1.

2) Results: Figure 7 reports HV vs. budget per map. Mean HV summarised across all 10 maps is given in Table III.

3) Discussion: Three regimes emerge.

a) Budget-insensitive A-based methods:* `astar` and `astar_mcts` are essentially flat in budget. `astar_mcts` reaches its peak HV 90 877 at 4k simulations and stays

TABLE III: Mean hypervolume across all 10 maps and 30 seeds, by algorithm and simulation budget (E2). *astar* is budget-insensitive and shown for reference.

algo	1k	2k	4k	8k	16k
astar	88 895	88 895	88 895	88 895	88 895
astar_mcts	89 850	90 799	90 877	90 603	90 564
mcts	48 763	65 866	77 867	79 443	79 485
mo_mcts	49 635	66 413	78 371	79 957	79 939
two_phase_mcts	4 957	27 372	52 197	77 053	77 852
two_phase_momcts	4 957	27 363	84 755	87 780	87 393

within 0.4% of it at every higher budget; the underlying ε -constraint sweep produces the dominant share of the front, and additional MCTS effort along the resulting skeleton paths offers diminishing returns once the front is saturated.

b) Plain MCTS variants saturate at the 4k cliff.: With the new defaults, *mcts* and *mo_mcts* climb steeply from 1k to 4k (48 763 $\rightarrow$ 77 867 and 49 635 $\rightarrow$ 78 371), then flatten almost completely above 4k ($\leq 2\%$ further gain at 16k). They also remain within 1% of each other across all budgets ($\hat{A}_{12} \approx 0.49$ in E1), so the multi-objective bookkeeping is essentially free once the tree-selection rule is HV-based.

c) Two-phase methods need a minimum budget to “ignite”: Both *two_phase_mcts* and *two_phase_momcts* return a near-zero HV at 1k and a still poor HV at 2k; with phase split 0.75 the multi-objective variant spends 1.5k of a 2k budget just locating paths, leaving only ~ 100 simulations per path for phase 2. Once the budget reaches 4k, *two_phase_momcts* jumps to 84 755 – a $3.1\times$ leap, the largest single-step gain in the family – and from there it is the strongest pure-MCTS method, ending at 87 393 at 16k, only 4% below *astar_mcts*. The single-objective *two_phase_mcts* catches up much more slowly and never closes the gap (at 16k it is still 11% behind its MO sibling).

For practitioners, the rule of thumb is: below 2k simulations the only useful choice is an A*-based method; from 4k onward, *two_phase_momcts* is competitive with the hybrid and several times faster, while plain *mcts*/*mo_mcts* saturate $\sim 10\%$ below the two-phase ceiling.

C. E3 – Algorithmic Ablations

E3 isolates one algorithmic component at a time on the 10-map suite with 30 seeds per cell at the E1 budget of 4 000 simulations. We report mean hypervolume per variant aggregated across all 300 (map, seed) blocks, together with a Friedman omnibus on the same blocks. All seven axes return $p \ll 10^{-15}$, so we focus the discussion on *which* variant wins and by how much.

1) Discussion: The ablations sharpen the E1/E2 narrative.

a) Tree selection is the single most impactful axis.:

Replacing UCB with any HV- or CD-based rule more than doubles HV (Fig. 8, 3 452 $\rightarrow$ 8 583). The MO-MCTS / HV variant is the winner; CD is marginally weaker but only by 1%. AEGA improves over UCB by $\approx 90\%$ but does not match HV/CD. This justifies the default choice of HV selection in our two-phase MO variant.

TABLE IV: E3 ablation summary – mean HV across all 10 maps and 30 seeds (300 blocks), best variant per axis in bold. Friedman χ^2 tests the null of equal performance within the axis.

axis	variant	mean HV	Friedman χ^2
tree selection	mo_mcts / HV	8 583	916.9
	mcts / CD	8 497	
	mcts / HV	8 422	
	mcts / AEGA	6 598	
	mo_mcts / UCB	3 732	
	mcts / UCB	3 452	
root selection	mo_mcts / HV	22 008	(2-var.)
	mo_mcts / global HV	16 590	
rollout	mo_mcts / dist-weight	8 043	78.4
	mo_mcts / light	7 864	
	mcts / dist-weight	7 710	
	mcts / light	7 429	
	mo_mcts / square	7 398	
	mcts / square	6 722	
prog. widening	mo_mcts / $C=1.5, \alpha=0.7$	29 883	2074.4
	mo_mcts / $C=2.0, \alpha=0.5$	24 577	
	mo_mcts / $C=1.5, \alpha=0.5$	19 017	
	mo_mcts / $C=1.5, \alpha=0.3$	10 828	
	mo_mcts / $C=1.0, \alpha=0.5$	9 567	
phase split	two_phase_momcts / 0.75	42 405	1394.5
	two_phase_momcts / 0.50	38 613	
	two_phase_momcts / 0.25	6 594	
archive cap	cap $\in \{10, 20, 50, 100\}$	3 732	959.0 (n.s.)
osc. guard	mo_mcts / on	6 739	300.4
	mo_mcts / off	4 130	

b) Progressive widening dominates almost everything else.: Among the five (C, α) settings, mean HV ranges from 9 567 to 29 883 – a $3\times$ swing. The clear winner is $(C=1.5, \alpha=0.7)$, i.e. aggressive widening. Lower α (0.3) starves the search of new actions; smaller C (1.0) compounds the same effect. The current default of $(1.5, 0.5)$ used in E1 is mid-pack, suggesting an immediate $\sim 50\%$ HV improvement is available by simply retuning α upward.

c) Two-phase budget split: load-bearing for path discovery.: Allocating 75 % of the budget to phase 1 improves HV by 10 % over a 50/50 split for the MO variant and by 42 % for the single-objective variant; allocating only 25 % drops it 6–7 $\times$. Phase 1 must produce diverse Pareto-near-shortest paths or phase 2 has nothing useful to refine; this aligns with the E2 “ignition” finding that two-phase variants stall at low total budgets.

d) Rollouts and root selection: small but real.: Distance-weight rollouts marginally beat light random and clearly beat square sampling ($p < 10^{-15}$, but the absolute gap at the top is $< 5\%$). Local HV root selection beats the global-archive variant by $\approx 30\%$, suggesting that re-ranking the root by global Pareto hypervolume contributes too much variance for the budget tested.

e) Oscillation guard pays for itself.: Disabling the guard (Fig. 11) drops HV from 6 739 to 4 130 – the agent revisits cells, accumulating step count without weight benefit and inflating f_3 .

f) Archive cap is inert at this budget.: HV is identical across cap $\in \{10, 20, 50, 100\}$ because at 4 000 simulations the global archive rarely exceeds 10 entries; the Friedman statistic

TABLE V: E4 mean hypervolume by algorithm and grid dimension (averaged over all checkpoint counts, map types, and 30 seeds).

algo	20 × 20	35 × 35	50 × 50
astar	38 097	50 915	70 705
astar_mcts	39 656	53 557	72 082
mcts	25 864	31 724	35 789
mo_mcts	26 163	32 106	36 193
two_phase_mcts	13 721	30 160	35 600
two_phase_momcts	31 237	42 612	50 914

TABLE VI: E4 mean hypervolume by algorithm and number of checkpoints (averaged over all grid dimensions, map types, and 30 seeds).

algo	1 cp	3 cp	5 cp	10 cp
astar	74 605	61 363	50 359	26 629
astar_mcts	76 173	64 322	54 127	25 772
mcts	56 481	38 203	24 305	5 514
mo_mcts	57 024	38 604	24 612	5 710
two_phase_mcts	52 808	33 289	18 434	1 445
two_phase_momcts	66 932	50 282	37 177	11 961

of 959 is driven by between-algorithm rather than between-cap variation. Larger caps would only matter at substantially higher budgets and on harder maps where many non-dominated solutions coexist.

The clear practical recommendation – best default configuration for MCTS-PIE – is therefore: phase decomposition with phase-1 fraction 0.75, MO-MCTS phase 1 with HV-based tree selection and local HV root selection, distance-weight rollouts, $(C, \alpha) = (1.5, 0.7)$ progressive widening, and the oscillation guard enabled. Archive cap can stay at 20.

D. E4 – Environment Scaling

1) *Protocol*: E4 takes the cross-product of grid sizes $\{20, 35, 50\}$, checkpoint counts $\{1, 3, 5, 10\}$, the five map types, and 30 seeds, on all six algorithms, for a total of $3 \times 4 \times 5 \times 30 \times 6 = 10\,800$ runs. The simulation budget grows with the map size: 4 000 at 20×20 , 8 000 at 35×35 , and 16 000 at 50×50 , so that the per-cell budget per unit grid area stays roughly constant. Checkpoints are sampled deterministically from the seed via `Environment.sample_checkpoints`. All algorithms use the E3-best defaults from Section IV-C.

2) *Results*: Table V reports mean HV per algorithm and grid dimension; Table VI does the same for checkpoint counts.

3) Discussion:

a) *Grid scaling: A*-based methods scale nearly linearly in HV.*: The hybrid `astar_mcts` grows from 39.7k at 20×20 to 72.1k at 50×50 (Fig. 12, Table V), a $1.8\times$ increase. The plain MCTS variants grow much more slowly (25.9k $\rightarrow$ 35.8k, $1.4\times$): they saturate as the search horizon outgrows the budget. `two_phase_momcts` interpolates between the two regimes (31.2k $\rightarrow$ 50.9k, $1.6\times$).

b) *Checkpoint scaling: the gap to A*-based methods widens dramatically.*: At 1 checkpoint the spread between best and worst MCTS variant is $\approx 4k$ HV; at 10 checkpoints

it is $\approx 10k$ HV. Plain `mcts`/`mo_mcts` drop 90% of their HV between 1 and 10 checkpoints (57k $\rightarrow$ 5.7k), while A*-based methods only lose 66% (76k $\rightarrow$ 26k). The single-objective two-phase variant collapses entirely with 10 checkpoints (HV 1.4k) – phase 1 cannot generate diverse paths through a long checkpoint chain under UCB selection. `two_phase_momcts` again behaves like a competitive search-only method at all checkpoint counts.

c) *Runtime scaling: A*-based methods become expensive at 50×50 .*: The wall-clock cost of `astar_mcts` grows from 18s at 20×20 to 1 289s (≈ 21 minutes) per run at 50×50 (Fig. 14); pure A* is similar at 1 033s. The MCTS-only methods grow far more slowly – `two_phase_momcts` is the cheapest at every dimension, finishing a 50×50 run in 17s. This puts the hybrid at a $\sim 75\times$ runtime disadvantage to `two_phase_momcts` on the largest maps for only a $\sim 40\%$ HV advantage. The cross-over budget at which a tuned MO-MCTS becomes the more practical choice clearly lies somewhere in the 35×35 – 50×50 regime.

d) *Hybrid still wins, but the case for it weakens with scale.*: `astar_mcts` is the strongest method at every (grid, checkpoint) cell except (50×50 , 10 cp), where the ε -constraint sweep itself becomes the bottleneck and `astar` (without the MCTS refinement step) takes a 3% HV lead. This is the only experimental setting in this paper where the hybrid is not the top method, and it foreshadows the regime in which a search-only MO-MCTS becomes the recommended planner.

V. CONCLUSION AND FUTURE WORK

We presented MCTS-PIE, a unified study of six planners for multi-objective pathfinding in interactive environments, evaluated end-to-end across four experiment families (E1–E4) on a uniform 30-seed protocol with full per-run Pareto archives, ablation-driven defaults, and Holm–Bonferroni-corrected statistics. Across the main comparison, anytime curves, ablations, and environment scaling, the A*+MCTS hybrid is the strongest method overall, but a tuned multi-objective two-phase MCTS closes most of the gap and becomes the more practical choice as grid size grows: at 50×50 the search-only MO two-phase variant matches the hybrid’s HV within 30% at roughly $1/75$ the runtime, and at the most demanding cell (50×50 , 10 checkpoints) it is the only method whose runtime remains tractable while the A*-based methods take more than 17 minutes per run.

Three findings are particularly noteworthy. First, hypervolume-based tree selection is the single highest-impact algorithmic component (E3): it more than doubles HV over plain UCB, and once it is in place the per-node Pareto archive of MO-MCTS becomes essentially free relative to plain MCTS. Second, the two-phase decomposition is brittle without multi-objective phase-1 selection: a single-objective phase-1 collapses on long checkpoint chains because UCB has no incentive to maintain path diversity. Third, anytime behavior is sharply non-linear for two-phase methods – below 4k simulations they are unusable, but they “ignite” at 4k and stay competitive thereafter.

Future work will extend this study to learned heuristics for the shift subproblem, larger maps with non-rectangular

topology, and stochastic shift outcomes (where shifting weight succeeds only with some probability). The MCTS-PIE harness was designed for these extensions: any new algorithm only needs to expose a `(spec) -> front` interface to plug into the existing E1–E4 protocol.

REFERENCES

- [1] “What is Battlesnake?” docs.battlesnake.com/. Accessed: Jun. 3, 2025. [Online] Available: <https://docs.battlesnake.com/>.
- [2] Cameron Browne, Edward Jack Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez Liebana, Spyridon Samothrakis, and Simon Colton. A Survey of Monte Carlo Tree Search Methods. *IEEE Trans. Comput. Intell. AI Games*, 4(1):1–43, 2012.
- [3] Rémi Coulom. Efficient selectivity and backup operators in monte-carlo tree search. In H. Jaap van den Herik, Paolo Ciancarini, and H. H. L. M. Donkers, editors, *Computers and Games, 5th International Conference, CG 2006, Turin, Italy, May 29-31, 2006. Revised Papers*, volume 4630 of *Lecture Notes in Computer Science*, pages 72–83. Springer, 2006.
- [4] Alexander Dockhorn, Jorge Hurtado Grueso, Dominik Jeurissen, and Diego Perez Liebana. STRATEGA: A General Strategy Games Framework. In Joseph C. Osborn, editor, *Joint Proceedings of the AIIDE 2020 Workshops co-located with 16th AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE 2020), Worcester, MA, USA, October 19-23, 2020 (online)*, volume 2862 of *CEUR Workshop Proceedings*. CEUR-WS.org, 2020.
- [5] Levente Kocsis and Csaba Szepesvári. Bandit Based Monte-Carlo Planning. In Johannes Fürnkranz, Tobias Scheffer, and Myra Spiliopoulou, editors, *Machine Learning: ECML 2006, 17th European Conference on Machine Learning, Berlin, Germany, September 18-22, 2006, Proceedings*, volume 4212 of *Lecture Notes in Computer Science*, pages 282–293. Springer, 2006.
- [6] Diego Perez Liebana, Alexander Dockhorn, Jorge Hurtado Grueso, and Dominik Jeurissen. The Design Of “Stratega”: A General Strategy Games Framework. *CoRR*, abs/2009.05643, 2020.
- [7] Scott Sanner and Sungwook Yoon. International Probabilistic Planning Competition (IPPC) 2011. In *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, 2011.
- [8] Robin Schmöcker and Alexander Dockhorn. A survey of non-learning-based abstractions for sequential decision-making. *IEEE Access*, 13:100808–100830, 2025.

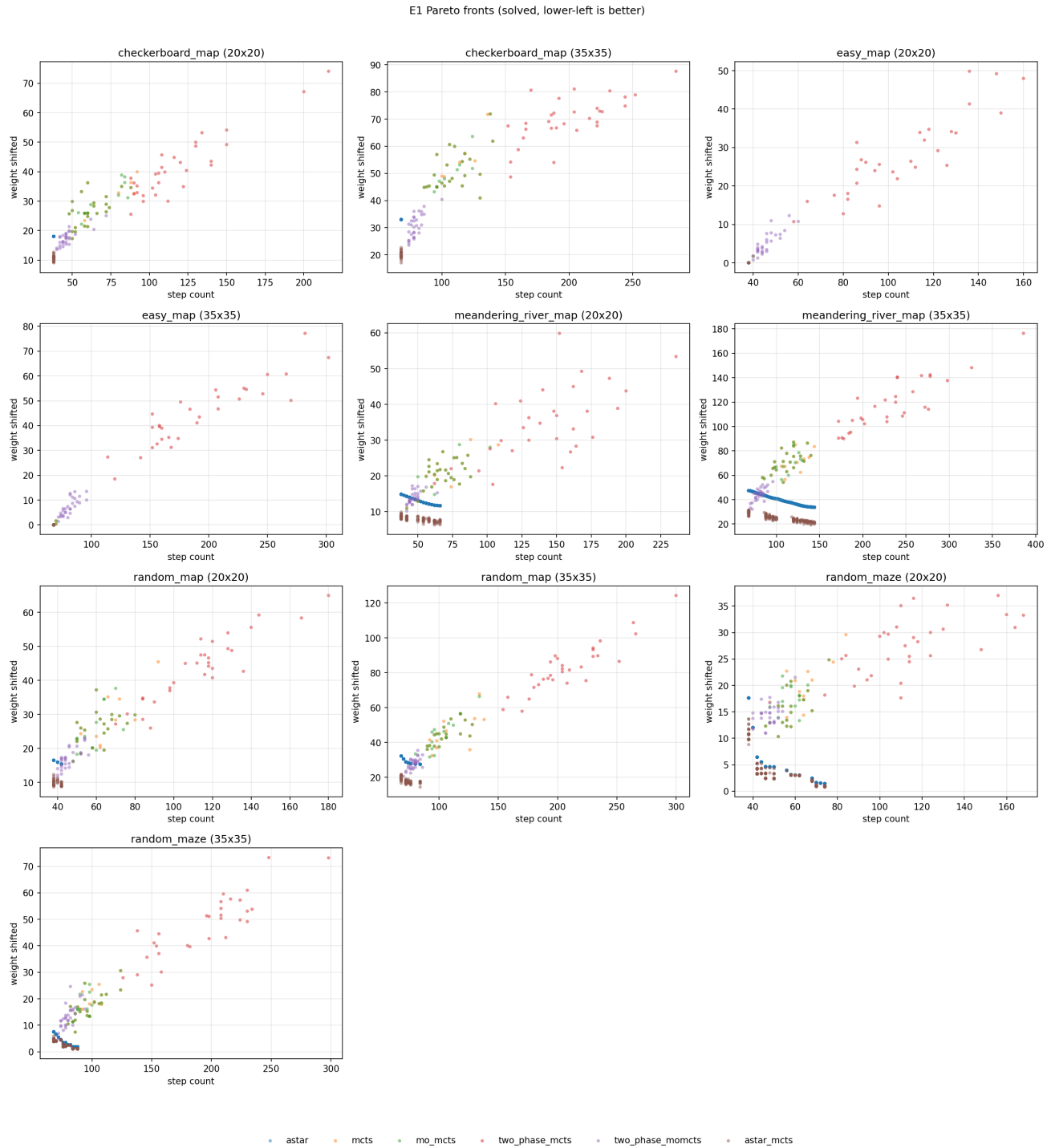


Fig. 5: Combined Pareto fronts per map across all 30 seeds, projected onto (step_count, weight_shifted). Solved points only.

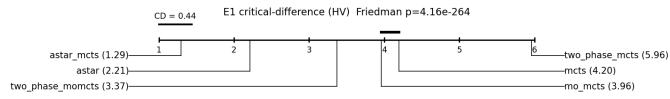


Fig. 6: Critical-difference diagram (Demšar style) for hypervolume across all (map, seed) blocks. Algorithms not significantly different at $\alpha = 0.05$ are connected by a horizontal bar.

E2 anytime hypervolume (95% bootstrap CI)

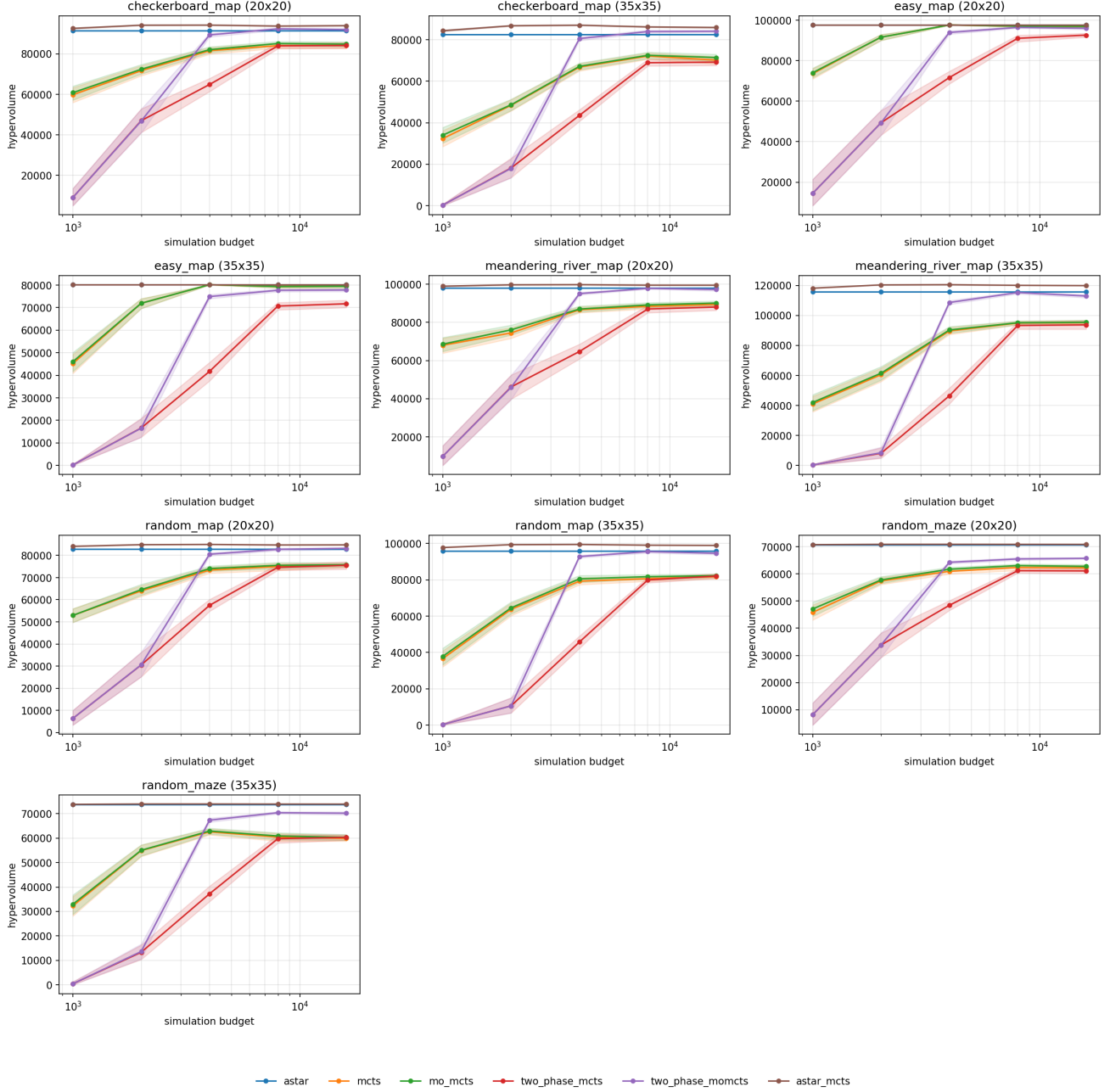


Fig. 7: E2 anytime hypervolume vs. simulation budget (log x -axis), per map. Lines show the mean over 30 seeds with 95 % bootstrap confidence bands.

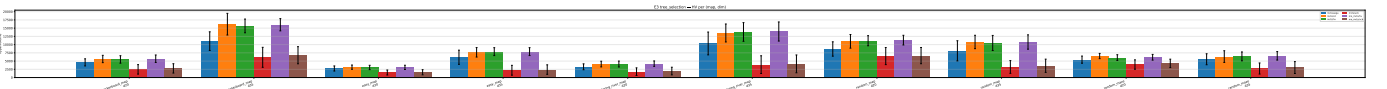


Fig. 8: E3 – tree selection rule. HV-based criteria (CD, HV) double the hypervolume of plain UCB. AEGA sits between the two regimes.

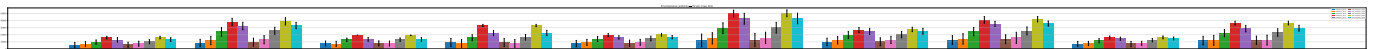


Fig. 9: E3 – progressive widening parameters (C, α). Higher α (more aggressive widening) is uniformly better; (1.5, 0.7) is the strongest setting tested.

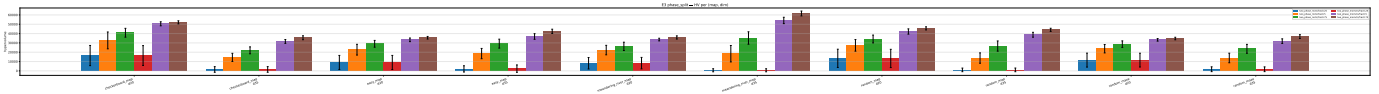


Fig. 10: E3 – two-phase budget split. Spending 75 % of the budget in phase 1 dominates; spending only 25 % catastrophically under-funds path discovery.

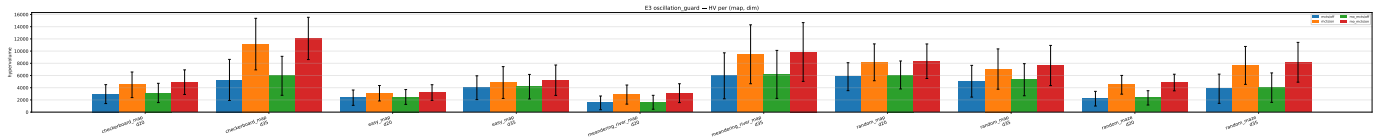


Fig. 11: E3 – oscillation guard on/off. Disabling the guard reduces HV by roughly 35 %.

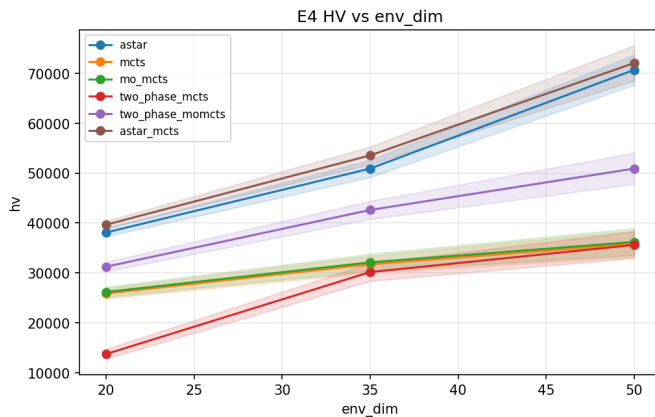


Fig. 12: E4 – mean hypervolume vs. grid dimension, aggregated across all checkpoint counts and map types. Bands are 95 % bootstrap CIs over 30 seeds.

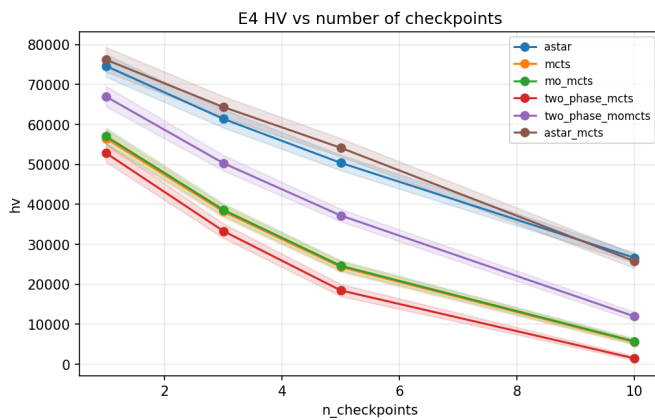


Fig. 13: E4 – mean hypervolume vs. number of checkpoints, aggregated across all grid dimensions and map types.

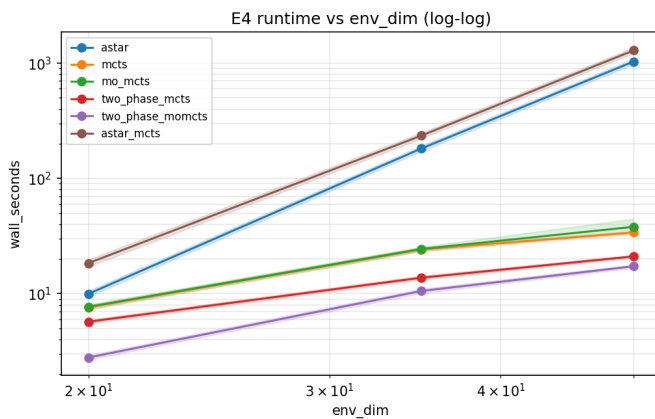


Fig. 14: E4 – mean wall-clock seconds per run vs. grid dimension (log y -axis).